

Unit-1: Concepts of NoSQL: MongoDB**1.1 Concepts of NoSQL. Advantages and Features.****1.1.1 MongoDB Datatypes (String, Integer, Boolean, Double, Arrays, Objects)****1.1.2 Database creation and dropping database****1.2 Create and Drop collections****1.3 CRUD operations (Insert, update, delete, find, Query and Projection operators)****1.4 Operators (Projection, update, limit(), sort()) and Aggregation commands****Concepts of NoSQL. Advantages and features.**

As the amount of data produced in the digital era explodes, databases are crucial to storing, managing, and analysing large amounts of data. Broadly, there are two types of databases: SQL and NoSQL databases.

SQL databases use a structured format, and the data is organized into tables with schemas. Unlike SQL databases, NoSQL databases offer flexible schema to store semi-structured and unstructured data. They provide the necessary infrastructure to manage and leverage the vast data generated in today's digital landscape. MongoDB, a popular NoSQL database. We can perform CRUD (Create, Retrieve, Update, and Delete) operations.

What is NoSQL?

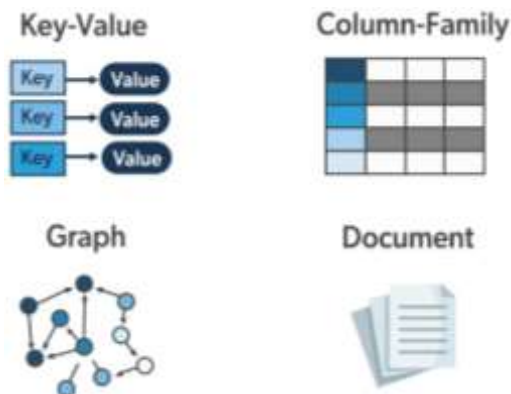
1. NoSQL stands for "Not Only SQL/non sql" and was created to address the limitations in relational databases. Relational databases follow a rigid structure, have a tabular format, and use SQL (Structured Query Language) for data manipulation.
2. NoSQL databases offer a more flexible approach for storing semi-structured and unstructured data.
3. The key advantage of NoSQL databases is that they are suitable for handling large volumes of data with varying structures. NoSQL databases are schema-less, allowing for greater liveliness and scalability.
4. However, some drawback They have weak consistency and limited support for complex queries. Despite these limitations, they're widely used for the benefits they offer and complement the SQL database

SQL vs NoSQL:

SQL	NoSQL
RELATIONAL DATABASE MANAGEMENT SYSTEM (RDBMS)	Non-relational or distributed database system.
These databases have fixed or static or predefined schema	They have dynamic schema.
These databases are not suited for hierarchical data storage.	These databases are best suited for hierarchical data storage.
These databases are best suited for complex queries	These databases are not so good for complex queries
Vertically Scalable	Horizontally scalable
Follows ACID property (Atomicity, Consistency, Isolation, and Durability) https://www.youtube.com/watch?v=GAe5oB742dw	Follows CAP(consistency, availability, partition tolerance)
Examples: MySQL, PostgreSQL, Oracle, MS-SQL Server etc	Examples: MongoDB, GraphQL, Redis, HBase, Neo4j, Cassandra, CouchDB etc

Types of NoSQL Databases

To cater to the needs of varying use cases, NoSQL databases use a variety of data models for organizing and accessing the data. Here are 4 common types of NoSQL databases.



1. **Key-value pair-based** is the simplest form of NoSQL databases. The data is stored as key and value pairs and can be retrieved using the key. Some examples are Redis and Amazon DynamoDB.
2. **Column-oriented** databases organize data in columns rather than in rows. This structure is efficient in retrieving data by columns. Apache Cassandra, HBase, and Google Bigtable are column-oriented.
3. **Graphs-based** databases store data in graph structures consisting of nodes, edges, and properties. Neo4j and Amazon Neptune are popular graph-based databases.
4. **Document-oriented** store data as JSON-like documents. The structure of a document can vary and even contain nested data. This is a widely used NoSQL database. Examples include MongoDB and CouchDB.

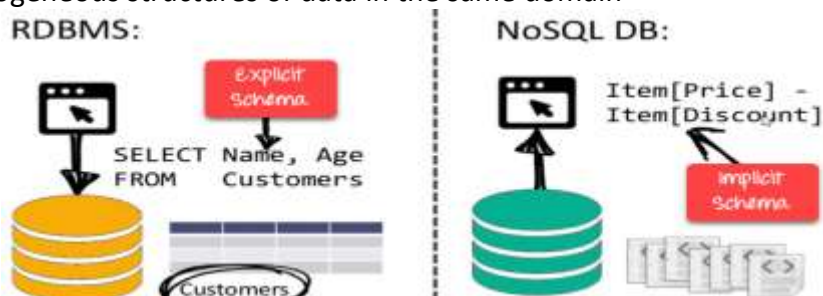
Features of NoSQL

Non-relational

- NoSQL databases never follow the relational model
- Never provide tables with flat fixed-column records
- Doesn't require object-relational mapping and data normalization
- No complex features like query languages, query planners, referential integrity joins, ACID

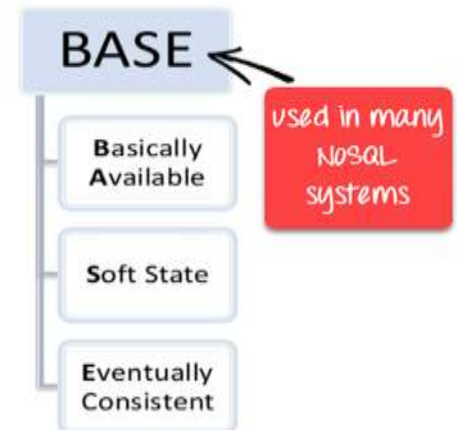
Schema-free

- NoSQL databases are either schema-free or have relaxed schemas
- Do not require any sort of definition of the schema of the data
- Offers heterogeneous structures of data in the same domain



The BASE Principle For Transaction

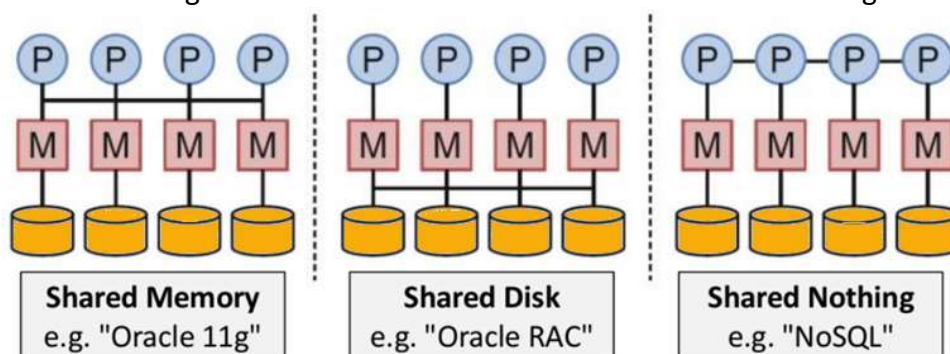
- The base is to NoSql as what ACID is to SQL.
- It ensures that NoSQL databases ensure their reliability despite the loss of consistency.
- The base stands for Basically Available Soft-state, eventually consistent.
- Eventually consistent – System can become eventually consistent, information is updated wherever necessary.

**Simple API**

- Offers easy to use interfaces for storage and querying data provided
- APIs allow low-level data manipulation & selection methods
- Text-based protocols mostly used with HTTP REST with JSON
- Mostly used no standard based NoSQL query language
- Web-enabled databases running as internet-facing services

Distributed

- Multiple NoSQL databases can be executed in a distributed fashion
- Offers auto-scaling and fail-over capabilities
- Often ACID concept can be sacrificed for scalability and throughput
- Mostly no synchronous replication between distributed nodes Asynchronous Multi-Master Replication, peer-to-peer, HDFS Replication
- Only providing eventual consistency
- Shared Nothing Architecture. This enables less coordination and higher distribution.

**Auto Balancing**

- Division of data among multiple servers automatically, with no assistance required from applications.

Integrated Caching

- NoSQL database cache data in system memory to increase data throughput and increase the performance in advance.
- High scalability, reliability with a simple data model and simple query language.

Advantages of NoSQL

- They have higher scalability.
- They use distributed computing.
- They are cost effective.
- They support flexible schema.
- They can process both unstructured and semi-structured data.
- There are no complex relationships, such as the ones between tables in an RDBMS.

Disadvantages of NoSQL

- No standardized rules
- Limited query capabilities
- RDBMS databases and tools are comparatively mature
- It does not offer any traditional database capabilities, like consistency when multiple transactions are performed simultaneously.
- Doesn't work as well with relational data
- The learning curve is stiff for new developers
- **Backup:** Some NoSQL databases, such as MongoDB, have a significant backup flaw. MongoDB does not include a method for backing up data in a consistent manner.
- **Management challenge:** Data administration in a NoSQL database is far more complicated than it is in a relational database.
- **Large document size:** Data is stored in JSON format in some database systems, such as MongoDB and CouchDB. This means that documents are quite huge (due to BigData, network bandwidth, and speed), and having detailed key names actually harms the document size by increasing it.
- **GUI is not available:** In the market, there are no flexible GUI mode tools for accessing the database.
- **Narrow focus:** NoSQL databases have a very restricted focus because they are primarily intended for storage and offer extremely little functionality.

MongoDB: An introduction

MongoDB is a powerful, open-source NoSQL database that offers a document-oriented data model, providing a flexible alternative to traditional relational databases. Unlike SQL databases, MongoDB stores data in BSON (which is more faster than JSON ,support more datatype in Binary JSON) format, which is similar to JSON, enabling efficient and scalable data storage and retrieval.

What is MongoDB?

MongoDB the most popular NoSQL database, is an open-source document-oriented database. The term 'NoSQL' means 'non-relational'. This means that MongoDB isn't based on a table-like relational database structure but provides an altogether different mechanism for the storage and retrieval of data. This format of storage is called BSON .

A simple MongoDB document Structure:

```
{
  title: 'Test',
  by: 'admin',
  url: 'https://www.ckpcmc.org',
  type: 'NoSQL'
}
```

- SQL databases store data in tabular format. This data is stored in a predefined data model which is not very much flexible for today's real-world highly growing applications.
- Modern applications are more networked, social and interactive than ever. Applications are storing more and more data and are accessing it at higher rates.
- Relational Database Management System(RDBMS) is not the correct choice when it comes to handling big data by the virtue of their design since they are not horizontally scalable. If the database runs on a single server, then it will reach a scaling limit.
- NoSQL databases are more scalable and provide superior performance. MongoDB is such a NoSQL database that scales by adding more and more servers and increases productivity with its flexible document model.

RDBMS vs MongoDB

- RDBMS has a typical schema design that shows number of tables and the relationship between these tables whereas MongoDB is document-oriented. There is no concept of schema or relationship.
- Complex transactions are not supported in MongoDB because complex join operations are not available.
- MongoDB allows a highly flexible and scalable document structure. For example, one data document of a collection in MongoDB can have two fields whereas the other document in the same collection can have four.
- MongoDB is faster as compared to RDBMS due to efficient indexing and storage techniques.
- There are a few terms that are related in both databases. What's called Table in RDBMS is called a Collection in MongoDB. Similarly, a Row is called a Document and a Column is called a Field. MongoDB provides a default '_id' (if not provided explicitly) which is a 12-byte hexadecimal number that assures the uniqueness of every document. It is similar to the Primary key in RDBMS.

MongoDB database features

- **Document Oriented:** MongoDB stores the main subject in the minimal number of documents and not by breaking it up into multiple relational structures like RDBMS. Data in MongoDB is stored in documents with key-value pairs instead of rows and columns, which makes the data more flexible when compared to SQL databases. For example, it stores all the information of a “student” in a single document called “student” and not in distinct relational structures like **Stud_per**, **stud_mark**, **stud_subject** etc.

- **Indexing:** Without indexing, a database would have to scan every document of a collection to select those that match the query which would be inefficient. So, for efficient searching Indexing is a must and MongoDB uses it to process huge volumes of data in very less time.
- **Scalability:** MongoDB scales horizontally using sharding (partitioning data across various servers). Data is partitioned into data chunks using the shard key (field name) and these data chunks are evenly distributed across shards that reside across many physical servers. Also, new machines can be added to a running database.
- **Replication and High Availability:** MongoDB increases the data availability with multiple copies of data on different servers. By providing redundancy, it protects the database from hardware failures. If one server goes down, the data can be retrieved easily from other active servers which also had the data stored on them.
- **Aggregation:** Aggregation operations process data records and return the computed results. It is similar to the GROUPBY clause in SQL. A few aggregation expressions are sum, avg, min, max, etc
- **Load balancing.** MongoDB handles load balancing without the need for a separate, dedicated load balancer, through either vertical or horizontal scaling.
- **Schema-less.** MongoDB is a schema-less database, which means the database can manage data without the need for a blueprint.

MONGODB DATA TYPES (STRING, INTEGER, BOOLEAN, DOUBLE, ARRAYS, OBJECTS)

In **MongoDB**, the documents are stored in **BSON** which is the binary encoded format of **JSON** and using BSON we can make remote procedure calls in MongoDB. BSON data format supports various data types.

1. String

This is the most commonly used data type in MongoDB to store data, BSON strings are of **UTF-8**. So, the **drivers** for each **programming language** convert from data types to the string format of the language to **UTF-8** while serializing and de-serializing BSON. The string must be a valid UTF-8.

Example: In the following example we are storing the name of the student in the student collection:

```
Command Prompt - mongo
switched to db gfg
> db.student.insertMany([{"name": "Akshay"}, {"name": "Vikash"}])
```

Here, the name field is a string representing the student's name.

2. Integer

In MongoDB, the integer data type is used to store an integer value. We can store integer data type in two forms 32-bit signed integer and 64-bit signed integer. These are used to store whole numbers, such as ages, counts, or any other numerical data that doesn't require decimal points.

Example: In the following example we are storing the age of the student in the student collection:

```

{
  "_id" : ObjectId("601af345b6d54aa34c9c6df5"),
  "name" : "Akash",
  "age" : 19
}

```

In this example, age is stored as an integer.

3. Double

The **double** data type is used for storing floating-point numbers (decimal values). It's commonly used for storing data that requires decimal precision, such as prices, percentages, or scores.

Example: In the following example we are storing the marks of the student in the student collection:

```

{
  "age" : 20,
  "marks" : 546.43
}

```

In this example, marks is stored as a double to store the decimal value.

4. Boolean

The **boolean** data type stores one of two values: **true** or **false**. It's used for representing binary states, such as "active/inactive" or "pass/fail."

Example: In the following example we are storing the final result of the student as pass or fail in boolean values.

```

{
  "name" : "vishal",
  "pass" : true
}

```

In this example, the passed field is a boolean value indicating whether the student passed or failed.

5. Null

The **null** data type stores a **null** value. This is useful when you want to represent the absence of data, such as an optional field that may not be set.

Example: student does not have a mobile number so the number field contains the value null.

```

{
  "name" : "Akash",
  "MobNo" : null,
  "skills" : [
    "c",
    "c++",
    "java",
    "python",
    "JS"
  ]
}

```

Here, phone_number is set to null, indicating that the student has not provided a phone number.

6. Array

The **array** data type allows us to store multiple values in a single field. MongoDB arrays can contain values of the same or different data types, providing flexibility in how you store collections of related data. In MongoDB, the array is created using square brackets([]).

Example: In the following example, we are storing the technical skills of the student as an array.

```

{
  "name" : "Akash",
  "MobNo" : null,
  "skills" : [
    "C",
    "C++",
    "java",
    "python",
    "JS"
  ]
}

```

In this example, the skills field stores an array of strings representing the student's technical skills.

7. Object (Embedded Document)

Object data type stores embedded documents. Embedded documents are also known as nested documents. Embedded document or nested documents are those types of documents which contain a document inside another document. Embedded documents allow us to structure our data hierarchically, which is useful for representing more complex data models.

Example: In the following example, we are storing all the information about a book in an embedded document.

```

{
  "_id" : ObjectId("601af71f6fd54aa34c9c6df9"),
  "Book" : {
    "name" : "C in depth",
    "writer" : "Aaksh"
  }
}
>

```

Here, the book field stores an embedded document with properties like title, author, and published.

8. Object Id

Whenever we create a new document in the collection MongoDB automatically creates a unique [object id](#) for that document(if the document does not have it). There is an `_id` field in MongoDB for each document. The data which is stored in Id is of hexadecimal format and the length of the id is 12 bytes which consist:

- 4-bytes for Timestamp value.
- 5-bytes for Random values. i.e., 3-bytes for machine Id and 2-bytes for process Id.
- 3- bytes for Counter

We can also create your own id field, but make sure that the value of that id field must be unique.

Example: In the following example, when we insert a new document it creates a new unique object id for it.

```

> db.book.find().pretty()
{
  "_id" : ObjectId("601af71f6fd54aa34c9c6df9"),
  "Book" : {
    "name" : "C in depth",
    "writer" : "Aaksh"
  }
}

```

In this example, MongoDB automatically generates a unique `_id` for each document.

9. Undefined

The **undefined** data type represents a value that is not defined. It is rarely used in modern MongoDB applications, as it has been replaced by the null value for most practical purposes.

Example: In the following example the type of the duration of the project is undefined.

```

> db.project.insertOne({name:"FauG",duration:undefined,binaryValue:"110011001"})
{
  "acknowledged" : true,
  "insertedId" : ObjectId("601af8516fd54aa34c9c6dfa")
}
> db.project.find().pretty()
{
  "_id" : ObjectId("601af8516fd54aa34c9c6dfa"),
  "name" : "FauG",
  "duration" : undefined,
  "binaryValue" : "110011001"
}

```

In this example, the duration field is set to undefined.

10. Binary Data

The **binary data** data type stores binary information such as images, files, or encrypted data. Binary data is often used when working with non-textual data.

Example: In the following example the value stored in the `binaryValue` field is of binary type.

```

{
  "_id" : ObjectId("601af8516fd54aa34c9c6dfa"),
  "name" : "FauG",
  "duration" : undefined,
  "binaryValue" : "110011001"
}

```

Here, the image field stores binary data.

11. Date

Date data type stores date. It is a 64-bit integer which represents the number of milliseconds. BSON data type generally supports UTC datetime and it is signed. If the value of the date data type is negative then it represents the dates before 1970. There are various methods to return date, it can be returned either as a string or as a date object. Some method for the date:

- **Date():** It returns the current date in string format.
- **new Date():** Returns a date object. Uses the `ISODate()` wrapper.
- **new ISODate():** It also returns a date object. Uses the `ISODate()` wrapper.

Example: In the following example we are using all the above method of the date:

```
>
> var date1 = Date()
> var date2 = new Date()
> var date3 = new ISODate()
> db.date.insertOne({Date1:date1,Date2:date2,Date3:date3})
{
  "acknowledged" : true,
  "insertedId" : ObjectId("601afa326fd54aa34c9c6d6c")
}
> db.date.find().pretty()
{
  "_id" : ObjectId("601afa326fd54aa34c9c6d6c"),
  "Date1" : "Thu Feb 04 2021 01:00:34 GMT+0530 (India Standard Time)",
  "Date2" : ISODate("2021-02-03T19:30:40.014Z"),
  "Date3" : ISODate("2021-02-03T19:30:56.454Z")
}
```

In this example, the created at field stores the current date and time.

CREATE A DATABASE IN MONGODB

In **MongoDB**, databases are not created explicitly until you insert some data into them.

The **use** command is employed to either create a **new database** or switch to an existing one.

Syntax:

use database_name

Example

use tyd

Explanation:

- If the **tyd** database doesn't exist, MongoDB will create it automatically once you insert data into it.
- If the database already exists, the **use** command simply switches the context to that database.
- Note that MongoDB doesn't actually create the database until you store some data.

CHECK EXISTING DATABASES:

To see all the databases that already exist in your MongoDB instance, use the **show dbs** command:

Syntax:

show dbs

CHECK CURRENT DATABASE NAME

To check the current database in use, we can use the **db** command:

Syntax:

db

This will return the name of the currently selected database, e.g., tyd

CRUD OPERATIONS (INSERT, UPDATE, DELETE, FIND, QUERY AND PROJECTION OPERATORS)

The following is a brief overview of what each operation does:

- **Create:** This is the operation that is used to insert or add new documents to the database.
- **Read:** This operation is used to view or fetch documents from the database.
- **Update:** This operation should be used when modifying existing documents.
- **Delete:** As the name suggests, the delete operation is used to remove documents from the database.

Together these four CRUD operations make up the essential operations of interacting with MongoDB server.

CREATE A COLLECTION IN MONGODB

In MongoDB, a collection is a **group of documents**. Collections are similar to tables in relational databases. However, collections do not enforce schema, allowing you to store documents with varying structures in the same collection. This flexibility is one of the **key features** of MongoDB.

To explicitly create a collection in MongoDB, you can use the **createCollection()** method. However, it's important to note that collections are also automatically created when you insert the first document into them.

Syntax:

```
db.createCollection('collection_name');
```

Example:

To create a collection called Student, you would use the following command:

```
db.createCollection('Student');
```

Explanation:

- The **createCollection()** method is used to explicitly create a collection in MongoDB.
- If you attempt to insert a document into a collection that doesn't exist, MongoDB will automatically create it for you without needing the **createCollection()** method.

For example, running the following command will automatically create the **Student collection** if it doesn't exist:

```
db.Student.insertOne({Name: "Om", age: 19})
```

Output



```
tyd> db.createCollection("student")
{ ok: 1 }
```

Note:

- Collections are created automatically when the first document is inserted, so using **createCollection()** is optional.
- MongoDB allows for collections to contain documents with different fields and data types, which provides flexibility when structuring your data.

INSERT DOCUMENTS INTO A COLLECTION IN MONGODB

MongoDB stores data in the form of documents, which are essentially **JSON-like objects**.

There are two primary methods for inserting documents:

1. db.collection.insertOne()
2. db.collection.insertMany()

1. insertOne() Method

The [insertOne\(\) method](#) in MongoDB is used to insert a single document into a collection. This is the simplest way to add a new record into a collection when you need to insert just one document at a time.

Syntax:

```
db.collection_name.insertOne({ key1: value1, key2: value2, ... });
```

Example:

```
db.Student.insertOne({"name":"raj","age":12})
```

Output:

```

tyd> db
tyd
tyd> db.Student.insertOne({name:"raj",age:12});
{
  acknowledged: true,
  insertedId: ObjectId('68558afb598d9adc6f748a61')
}
tyd>

```

Explanation:

- The insertOne() method adds a single document to the collection.

2. insertMany() method

The [insertMany\(\) method](#) in MongoDB allows you to **insert multiple documents** into a collection at once. This method is ideal when you need to add multiple records in one operation, improving performance compared to inserting documents one by one.

Syntax:

```
db.collection_name.insertMany([ { key1: value1, key2: value2, ... }, { key1: value1, key2: value2, ... }, ... ]);
```

Example:

Let's say we want to create a collection named **studnet** and insert two documents. The first document will have a name field with the value "raj" and age field with 12. The second document will have name as "ram" and age as 22 and so on.

```
db.Student.insertMany([ {name:"raj",age:12},
                        {name:"ram",age:22},
                        {name:"raman",age:19} ]);
```

Output:

```

tyd> db.Student.insertMany([ {name:"raj",age:12}, {name:"ram",age:22}, {name:"raman",age:19} ]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('68558cbe598d9adc6f748a61'),
    '1': ObjectId('68558cbe598d9adc6f748a64'),
    '2': ObjectId('68558cbe598d9adc6f748a65')
  }
}
tyd>

```

Explanation:

The **insertMany()** method accepts an array of **DOCUMENTS**.

- In this example, three documents are inserted, students named raj, ram and raman.

READ OPERATION/ FIND METHOD

- The Read operations are used to retrieve documents from the collection, or in other words, read operations is used to query a collection for a document.
- We can perform read operation using the following method provided by the MongoDB:
 - **db.collection.find()**
 - **Db.collection.pretty()**
 - **db.collection.findOne()**

The find() method**Syntax**

```
>db.COLLECTION_NAME.find({condition},{fields})
```

find() method will display all the documents in a non-structured way.

```
>db.Student.find()
```

The pretty() Method

To display the results in a formatted way, you can use pretty() method.

Syntax

```
>db.COLLECTION_NAME.find().pretty()
```

Example

```
>db.student.find().pretty()
```

Output:

```
[
  { _id: ObjectId('68558c0e598d9adc6f748a63'), name: 'raj', age: 12 },
  { _id: ObjectId('68558c0e598d9adc6f748a64'), name: 'ram', age: 22 },
  { _id: ObjectId('68558c0e598d9adc6f748a65'), name: 'raman', age: 19 }
]
```

Note: find() is a MongoDB method used to query documents in a collection. pretty() is a method used to format the query results in a more readable and structured manner.

The findOne() method

Apart from the find() method, there is **findOne()** method, that returns only one document.

Syntax

```
>db.COLLECTIONNAME.findOne({condition},{fields})
```

Example:

```
yd> db.Student.findOne({name:"raj"})
{ _id: ObjectId('68558c0e598d9adc6f748a5f'), name: 'raj', age: 12 }
```

UPDATE DOCUMENT

To update an existing document we can use the updateOne() or updateMany() methods.

The first parameter is a query object to define which document or documents should be updated. The second parameter is an object defining the updated data. Three different methods of updating documents:

- db.collection.updateOne()
- db.collection.updateMany()
- db.collection.replaceOne()

updateOne()

The updateOne() method will update the first document that is found matching the provided query.

Syntax:

- db.collection.update(<filter>, <update>, { options })

Parameters:

- <filter>: Query to match documents.
- <update>: Update operations using operators like \$set, \$inc, etc.
- { options } : Optional - upsert, multi (old), etc.

Operator	Description	Example
\$set	Set the value of a field (creates it if missing)	{ \$set: { status: "active" } }
\$unset	Remove the specified field	{ \$unset: { sal: "" } }

Operator	Description	Example
\$inc	Increment a numeric field by the given value	{ \$inc: { score: 5 } }
\$mul	Multiply a numeric field by the given value	{ \$mul: { price: 1.1 } }
\$rename	Rename a field	{ \$rename: { oldName: "newName" } }
\$min	Only update the field if the specified value is less than current	{ \$min: { lowScore: 40 } }
\$max	Only update the field if the specified value is greater than current	{ \$max: { highScore: 100 } }
\$currentDate	Set a field to the current date. Use <field>: true for a BSON date, or { \$type: "timestamp" }	{ \$currentDate: { lastModified: true } }
\$setOnInsert	Like \$set, but only when an upsert inserts a new document	{ \$setOnInsert: { createdAt: newDate() } }

example:

```
tyd> db.Student.updateOne({name:"raj"},{$set:{age:20}})
```

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Output



```
tyd> db.Student.find()
{ _id: ObjectId('68558ae598d9adc6f748a5f'), name: 'raj', age: 20 },
{ _id: ObjectId('68558ada598d9adc6f748a60'), name: 'raj', age: 12 },
{ _id: ObjectId('68558a7b598d9adc6f748a61'), name: 'raj', age: 12 }
```

updateMany()

The updateMany() method will update all documents that match the provided query.

Syntax:

```
Db.collectionname.updateMany({},{$set:{changevalue}})
```

Example:

```
tyd> db.Student.updateMany({},{$set:{age:22}})
```



replaceOne: Replaces an entire matching document with a new one.

```
db.Student.replaceOne(
  { name: "Amit" },
  { name: "Amit", age: 22, course: "MBA" }
);
```

DELETE DOCUMENT

The MongoDB shell provides the following methods to delete documents from a collection using `db.collection.deleteMany()` and `db.collection.deleteOne()`.

Delete Multiple Documents

To delete all documents from a collection, pass an criteria in filter document {} to the `db.collection.deleteMany()` method.

Syntax:

DB.COLLECTION_NAME.DELETEMANY({ KEY: VALUE })

Example:

```
db.studnet.deleteMany({ name: :'raj'})
```

Delete All Documents

To delete all documents from a collection, pass an empty filter document {} to the `db.collection.deleteMany()` method.

SYNTAX

```
db.collection name.deleteMany({ })
```

Delete Only One Document that Matches a Condition

To delete at most a single document that matches a specified filter use the `db.collection.deleteOne()` method.

Syntax:

```
db.collection_name.deleteOne({ key: value })
```

Example:

```
db.collection name.deleteOne({ name:'Raj' })
```

DROPPING THE COLLECTION

It is used to drop a collection from the database.

Syntax:

```
db.collection.drop()
```

example:

```
tyd> db.emp.drop()
true
```

drop() method will return true, if the selected collection is dropped successfully, otherwise it will return false.

VIEW EXISTING COLLECTIONS

After **creating collections** and **inserting data**, it's important to check the contents of your **current database** by listing all the collections. In MongoDB, you can view all the collections in your **current database** by using the **show collections command**.

Syntax:

show collections

Example:

If you're working in the **TYD** database and you want to see all the collections, simply run:
show collections

Output:

```
tyd> show collections
emp
prod
student
Student
tyd>
```

This shows that the Student collection exists within the tyd database.

MONGODB QUERY OPERATORS

- MongoDB offers different types of operators that can be used to interact with the database. Operators are special symbols or keywords that inform a compiler or an interpreter to carry out mathematical or logical operations.
- The query operators enhance the functionality of MongoDB by allowing developers to create complex queries to interact with data sets that match their applications.
- MongoDB offers the following query operator types:
 - Comparison
 - Logical
 - Element
 - Evaluation
 - Geospatial
 - Array
 - Bitwise
 - Comments

MongoDB operators can be used with any supported MongoDB command.

Comparison Operators

MongoDB comparison operators can be used to compare values in a document.

Operator	Description
\$eq	Matches values that are equal to the given value.
\$gt	Matches if values are greater than the given value.
\$lt	Matches if values are less than the given value.
\$gte	Matches if values are greater or equal to the given value.
\$lte	Matches if values are less or equal to the given value.

\$in	Matches any of the values in an array.
\$ne	Matches values that are not equal to the given value.
\$nin	Matches none of the values specified in an array.

Logical Operators

- MongoDB logical operators can be used to filter data based on given conditions. These operators provide a way to combine multiple conditions. Each operator equates the given condition to a true or false value.

Operator	Description
\$and	Joins two or more queries with a logical AND and returns the documents that match all the conditions.
\$or	Join two or more queries with a logical OR and return the documents that match either query.
\$nor	The opposite of the OR operator. The logical NOR operator will join two or more queries and return documents that do not match the given query conditions.
\$not	Returns the documents that do not match the given query expression.

MongoDB Projection

- MongoDB provides a special feature that is known as **Projection**. It allows us to select only the necessary data rather than selecting whole data from the document.
- This enable us to:
 - Project concise yet transparent data
 - Filter data without impacting the overall database performance
- We can use projection **with db.collection.find()** method.
- In this method, the second parameter is the projection parameter, which is used to specify which fields are returned in the matching documents.

```
db.{collection}.findOne({query}, {projection})
db.collection.find({}, {field1: value2, field2: value2, ..})
```

- If the value of the field is set to 1 it will be returned while displaying the document and if it is 0 it will not be displayed.
- There is no need to set _id field to 1 to return _id field, the find() method always return _id unless you set a _id field to 0.

MongoDB Projection Operators

- MongoDB projection method positively impacts database performance as it reduces the workload of the find query when trying to retrieve specific data from a document, minimizing resource usage.
- Ex:


```
Db.Student.find({criteria},{fieldname1,..})
      ○ Db.student.find({}, {name:1,city:1})
```
- To enhance the querying and reduce the workload, multiple operators can be used within a projection query like the ones below:
 - \$
 - \$elemMatch

- \$slice
- \$meta

- \$ (Positional Operator): Projects the first matched element in an array based on the query condition.

Syntax:

```
db.collection.find({ "arrayField.subField": <condition> }, { "arrayField.$": 1 })
```

Example: student

```
{
  "_id": 1,
  "name": "Amit",
  "scores": [
    { "type": "exam", "score": 90 },
    { "type": "quiz", "score": 80 },
    { "type": "homework", "score": 85 }
  ]
}
```

only the **first score with type "quiz"** for the student named **Amit**.

```
db.students.find(
  { name: "Amit", "scores.type": "quiz" },
  { "scores.$": 1 }
)
```

"scores.\$" tells MongoDB to include only the **first matched element** in the scores array that matches the condition "type": "quiz"

- \$elemMatch: The \$elemMatch projection operator is used to **project specific elements from an array** that **match a condition**. It is especially useful when **only one matching element** from an array is needed based on given criteria.

Syntax:

```
db.collection.find({query},{ arrayField: { $elemMatch: { <condition> } } })
```

Example:

- db.Student.insertOne({ "_id": 1, "name": "Amit", "scores": [{ subject: "Math", marks: 90 }, { subject: "Science", marks: 85 }, { subject: "English", marks: 75 }]});
- Get only the subject where marks > 80 and subject is "Science"

```
○ db.Student.find({name:"Amit"},{ scores: { $elemMatch: { subject:
  "Science", marks: { $gt: 80 } } } });
```

without \$elemMatch

```
○ db.Student.find({{ "scores. Subject": "Science", "scores.marks": { $gt: 80 }
  } });
```

- \$slice: Limits the number of array elements returned in the output.

Syntax:

```
db.collection.find({}, { arrayField: { $slice: N } })
```

Examples:

- First 3 elements: { comments: { \$slice: 3 } }
- Last 2 elements: { comments: { \$slice: -2 } }
- Start at index 1, return 2: { comments: { \$slice: [1, 2] } }

Example:

- `Db.student.insertOne({"_id": 1, "name": "Alice", "scores": [90, 85, 88, 95, 70]});`
- Get First 3 Scores
 - `db.students.find({ _id: 1 }, { scores: { $slice: 3 } })`
- Get Last 2 Scores
 - `db.students.find( { _id: 1 }, { scores: { $slice: -2 } })`
- Skip 1, Return Next 3 Scores
 - `db.students.find({ _id: 1 }, { scores: { $slice: [1, 3] } })`
- `$meta`: Returns metadata such as text search relevance score.
Syntax:
`db.collection.find({ $text: { $search: "term" } }, { score: { $meta: "textScore" } })`

Example:

```
db.emp.insertMany([
  {
    name: "Amit Desai",
    bio: "Amit is a brilliant computer science student specializing in databases and MongoDB."
  },
  { name: "Nita Patel",
    bio: "Nita is passionate about data analytics, Python, and AI."
  },
  { name: "Rahul Shah",
    bio: "Rahul is working on a MongoDB project involving student performance tracking."
  },
  {
    name: "Sneha Kapoor",
    bio: "Sneha loves web development using React and Node.js."
  }
])
```

Create index

```
db.emp.createIndex({name:"text"})
```

search meta score on the base of name

```
db.emp.find(
  { $text: { $search: "amit" } },
  {
    name: 1,
    score: { $meta: "textScore" }
  })
```

Aggregation

- Aggregation is a way of processing a large number of documents in a collection by means of passing them through different stages. The stages make up what is known as a **pipeline**.
- The stages in a pipeline can filter, sort, group, reshape and modify documents that pass through the pipeline.
- One of the most common use cases of Aggregation is to calculate aggregate values for groups of documents. This is similar to the basic aggregation available in SQL with the GROUP BY clause and COUNT, SUM and AVG functions.

How does MongoDB aggregation work?**Aggregation Pipelines**

- An aggregation pipeline consists of one or more stages that process documents:

- Each stage performs an operation on the input documents. For example, a stage can filter documents, group documents, and calculate values.
- The documents that are output from a stage are passed to the next stage.
- An aggregation pipeline can return results for groups of documents. For example, return the total, average, maximum, and minimum values.

MongoDB Aggregation Framework



- \$match stage – filters those documents we need to work with, those that fit our needs
- \$group stage – does the aggregation job
- \$sort stage – sorts the resulting documents the way we require (ascending or descending)
- The input of the pipeline can be a single collection, where others can be merged later down the pipeline.
- The pipeline then performs successive transformations on the data until our goal is achieved.

syntax

db.collectionName.aggregate(pipeline, options),

- collectionName – is the name of a collection,
- pipeline – is an array that contains the aggregation stages,
 - pipeline = [
 - { \$match : { ... } },
 - { \$group : { ... } },
 - { \$sort : { ... } }
 -]

\$match

- The \$match stage allows us to choose just those documents from a collection that we want to work with. It does this by filtering out those that do not follow our requirements.

\$project

- It is rare that you ever need to retrieve all the fields in your documents. It is good practice to return only those fields you need so as to avoid processing more data than is necessary.
- The \$project stage is used to do this and to add any calculated fields that you need.

\$group

- With the \$group stage, we can perform all the aggregation or summary queries that we need, such as finding counts, totals, averages or maximums.
- **MongoDB \$group aggregation operators**
 - The \$group stage supports certain expressions (operators) allowing users to perform arithmetic, array, boolean and other operations as part of the aggregation pipeline.

Operator	Meaning
\$count	Calculates the quantity of documents in the given group.

\$max	Displays the maximum value of a document's field in the collection.
\$min	Displays the minimum value of a document's field in the collection.
\$avg	Displays the average value of a document's field in the collection.
\$sum	Sums up the specified values of all documents in the collection.
\$push	Adds extra values into the array of the resulting document.

\$sort

- It sorts the resulting documents the way we want to display (in ascending or descending)

Example: Calculate total amount of Customers with Status 'A'.

- **\$skip** over the specified number of documents in the aggregation pipeline. Other options – optional parameters for the aggregation
- **\$limit[n]**: Limits the **number of documents** that pass through the aggregation pipeline.
 - Syntax: { \$limit: <number> }
 - Example: to display/to pass first 3 documents
`db.Student.aggregate([ { $limit: 3 } ])`
or
`db.Student.find().limit(3)`
- **\$skip[n]** : **\$skip** over the specified number of documents in the aggregation pipeline.
 - Syntax: { \$skip: <number> }
 - Example: Skip the **first 2 documents**:
`db.student.aggregate([ { $skip: 2 } ])`
or
`db.Student.find().skip(2)`
 - Example: Show **items 4 to 6**: (combine limit and skip)
`db.orders.aggregate([ { $skip: 3 }, { $limit: 3 } ])`

Sample Data in sales Collection:

```
db.createCollection("sales")
```

```
db.sales.insertMany([
{ "_id": 1, "item": "laptop", "category": "electronics", "price": 70000,
  "quantity": 2, "date": ISODate("2024-07-01T10:00:00Z")},
```

```
{ "_id": 2, "item": "laptop", "category": "electronics", "price": 70000, "quantity": 1,
  "date": ISODate("2024-07-03T10:00:00Z")},
```

```
{ "_id": 3, "item": "mobile", "category": "electronics", "price": 30000, "quantity": 5,
  "date": ISODate("2024-07-02T10:00:00Z")},
```

```
{ "_id": 4, "item": "chair", "category": "furniture", "price": 1500, "quantity": 10, "date":
  ISODate("2024-07-01T10:00:00Z")}]})
```

\$group: Total Quantity Sold per Item

```
db.sales.aggregate([{$group:{_id: "$item",totalQuantity:{$sum:"$quantity" }}}])
```

Output:

```
{ "_id": "laptop", "totalQuantity": 3 }
{ "_id": "mobile", "totalQuantity": 5 }
{ "_id": "chair", "totalQuantity": 10 }
```

\$project: Calculate Total Revenue per Sale

```
db.sales.aggregate([{$project: {item: 1,totalRevenue: { $multiply: ["$price", "$quantity"] }}}])
```

Output:

```
{ "item": "laptop", "totalRevenue": 140000 }
{ "item": "laptop", "totalRevenue": 70000 }
{ "item": "mobile", "totalRevenue": 150000 }
{ "item": "chair", "totalRevenue": 15000 }
```

\$match: Filter for Electronics Category Only

```
db.sales.aggregate([{$match: { category: "electronics" }}])
```

Output:

Returns only documents where category = electronics.

\$sort: Sort by Total Price Descending

```
db.sales.aggregate([
  {
    $project: {
      item: 1,
      totalPrice: { $multiply: ["$price", "$quantity"] }
    }
  },
  {
    $sort: { totalPrice: -1 }
  }
])
```

\$limit and \$skip: Pagination Example

Code:

```
db.sales.aggregate([
  { $sort: { date: -1 } },
  { $skip: 1 },
  { $limit: 2 }
])
```

Output:

Skips the latest record and gives the next 2.

\$group with Average and Max Revenue by Category

Code:

```
db.sales.aggregate([
  {
    $project: {
```

```
    category: 1,
    revenue: { $multiply: ["$price", "$quantity"] }
  },
  {
    $group: {
      _id: "$category",
      avgRevenue: { $avg: "$revenue" },
      maxRevenue: { $max: "$revenue" }
    }
  }
])
```